Compliance & Secure Coding Practices in R26-CS-010

Final Year Research Project

Project ID: R26-CS-010

Project Title: Privacy-Enhanced Synthetic Data Generation for Autonomous Vehicle Training

Student Details:

Names	Student IDs	Individual component
Priyadarshi S.A.S.D.	IT22309556	Privacy Integration
Yubitha S.	IT22541284	Secure Dataset Generation
Apeksha M.K.S.R.	IT22110220	Utility Evaluation
Wijesinghe R.D.P.T.	IT22066916	Risk and Compliance

Supervisor: Prof. Anuradha Jayakody

Co-Supervisor: Mr. Dinith Primal

Date of Submission: October 2026

B.Sc. (Hons) Degree in Information Technology, Specialized in Cyber Security

Sri Lanka Institute of Information Technology

1.Introduction

Our project, R26-CS-010, generates synthetic driving data to train autonomous vehicles while protecting privacy. It has four components: Privacy Integration (PI), Dataset Generation (DG), Utility Evaluation (UE) and Risk and Compliance (RC). PI adds differential privacy. DG encrypts every file and controls who can access it. UE checks file integrity and label quality. RC attacks the outputs and maps the controls to the GDPR [1] and the NIST Privacy Framework [2]. We could not assume the data was private just because it was synthetic. The simulator uses real statistics, and trained models can leak their training data. This document lists the standards, techniques and tools we used to test this.

2.Secure Coding Standards

We did not follow one secure coding standard in full. Our code is in Python, so the SEI CERT standards [3] for C and C++ did not apply. The pipeline has no web interface, so we skipped the web parts of the OWASP Secure Coding Practices [4]. Our work still covers the OWASP areas of input validation, cryptography, access control, error handling, data protection and configuration. The cryptography follows FIPS 197 [5], FIPS 180-4 [6] and SP 800-38D [7]. SP 800-57 [8] guided how we keep keys separate from data. RBAC uses least privilege and denies access by default. We checked the DG threat model with STRIDE. We did not use ISO/IEC 27001 [9] as a formal framework, but ISO/IEC 27559 [10] shaped the metadata scan in Attack 5. The team worked in two-week Scrum sprints, and each sprint ended with a supervisor review. Table 1 shows the security work in each phase.

Phase	Security and compliance activity
Requirements	DG wrote a threat model for an attacker who can reach storage. RC linked GDPR articles to controls before writing the attack scripts. PI used the proposal targets of $\varepsilon \leq 10$ and attack success $\leq 55\%$.
Design	PI added privacy at the point where real data enters the pipeline. DG moved from CBC to GCM. Security code, AES keys and HMAC keys were kept separate. Components share files under JSON Schema contracts.
Implementation	Keys load from environment variables or an untracked .env file. Settings are validated when they load. Decryption fails closed, and training stops at $\varepsilon = 10$.
Verification	GitHub Actions ran 107 unit tests on every push, along with pipeline and end-to-end tests. DG ran a five-test encryption audit. UE tested the poisoning detector with planted defects.
Audit	RC ran six independent attacks. PI produced HMAC-signed audit reports and a signed privacy certificate. RC wrote JSON and PDF reports mapped to GDPR and NIST.

Table 1: Security and compliance activity by lifecycle phase

3. Key Secure Coding Techniques Implemented with Examples

Table 2 is our checklist. Yes means implemented. Partial means implemented with a known gap. No means not implemented and N/A means not applicable.

Status	Technique	Example and evidence	Comp.
Input validation			
Yes	**Schema validation**	Every file passed between components is checked against a JSON Schema contract. A missing or wrongly typed field stops the run.	All
Yes	**Label validation**	The poisoning gate checks the field count, class ID, coordinates in [0, 1] and box area. A test copy with x_center = 1.5 and class 7 raised three flags. The real dataset (15,084 instances) passed.	UE
Yes	**Integrity checks**	SHA-256 digests are recomputed after decryption and compared with the town manifest. All 40,000 files matched.	DG, UE, RC
N/A	**Injection, XSS, TLS**	There is no database or web interface. Files move as encrypted files.	DG

Error handling and logging			
Yes	**Fail-closed decryption**	decrypt_and_verify checks the authentication tag first. If the check fails, no plaintext is written. Changing one byte caused a MAC failure.	DG
Yes	**Budget hard stop**	DPTrainingWrapper raises BudgetExceededError when the cumulative ε reaches 10. A test with a low limit confirmed this.	PI
Partial	**Logging**	Cumulative ε is logged at every step (3,090 rows per run). RBAC decisions are printed, but there is no tamper-evident access log.	PI, DG
Authentication and authorization			
Partial	**Role-based access**	The gateway denies access by default and has three roles. The audit notebook does not test denial cases yet.	DG
Yes	**Signed evidence**	Audit reports use HMAC-SHA256 with constant-time comparison. RC detected a copy where ε was edited from 0.990572 to 0.01.	PI, RC
No	**User authentication**	There is no password policy or multi-factor login. Roles are plain strings. This was out of scope and is listed as a limitation.	DG
Encryption and key handling			
Yes	**Authenticated encryption**	Each file uses AES-256-GCM with a random 16-byte nonce and a 16-byte tag. All 30 repeated encryptions gave different nonces. None of the 6,093 captured files had a readable PNG header.	DG
Partial	**Delivered dataset**	The five-town dataset uses AES-256-CBC, which cannot detect tampering. Integrity therefore depends on the SHA-256 manifests. Moving to GCM is future work.	DG, UE
Yes	**Secret handling**	Keys come from environment variables or an untracked .env file. A code review found keys in source files. We replaced them, and a regression test now checks for this.	All
No	**Key management**	There is no key service, key rotation or per-batch keys. The audit notebook uses a fixed test key so the results can be repeated.	DG
Sensitive files and privacy-preserving computation			
Yes	**Restricted files**	calibration_audit_log.json holds the noise for each query and goes only to RC. Subtracting it from the published values would reveal the source values.	PI
Partial	**Secure noise source**	Opacus secure mode is off because torchcsprng needs PyTorch 1.8.1. The signed certificate lists this.	PI
Yes	**Differential privacy**	Laplace noise uses $\varepsilon = 1.8$ per query over seven queries. DP-SGD uses $C = 1.0$, $\delta = 10^{-5}$ and a Rényi accountant. Membership inference AUC fell from 0.639 to 0.504 to 0.515.	PI

Table 2: Secure coding checklist. PI = Privacy Integration, DG = Dataset Generation, UE = Utility Evaluation, RC = Risk and Compliance.

4. Tools & Technologies Used for Secure Coding

The security code is written in Python. PyCryptodome [11] handles AES, hashlib handles SHA-256 and hmac handles HMAC-SHA256. DP-SGD runs on Opacus 1.4.1 [12]. JSON Schema and one config.yaml control the interfaces. pytest and GitHub Actions run the tests on every push. We did not use commercial static analysis tools (SonarQube, Fortify, Checkmarx) or web penetration testing tools (OWASP ZAP, Burp Suite), since there is no web interface. Instead, we wrote six attack scripts, a five-test encryption audit and a label poisoning detector. Static analysis and dependency scanning are not in our CI yet, and we suggest adding them.

5. Compliance Standards

The relevant rules are the GDPR [1] (Articles 5, 17, 25 and 32), the NIST Privacy Framework [2] and the NIST AI RMF [13]. HIPAA and PCI DSS do not apply because we hold no health or payment data. Table 3 links each GDPR article to a control and to the attack that tested it.

GDPR article	Project control	Verification evidence	Result
Art. 5(1)(b) Purpose limitation	Calibration uses seven aggregate statistics only	Calibration log shows seven aggregate queries and no individual records	Pass
Art. 5(1)(c) Data minimization	Seven aggregate fields, each with Laplace noise	Attack 3: noise ratio 0.94 (accepted range 0.5 to 1.5). Attack 6: 0% re-identification in 600 samples	Pass
Art. 5(2) Accountability	HMAC-SHA256 signed audit reports	Attack 1: 4 of 4 reports verified, tampering detected	Pass
Art. 17 Erasure	No individual records stored	Attack 5: no findings in 10,015 metadata files	Pass
Art. 25 Privacy by design	DP at calibration and training	Attack 2: reported ε matches logged ε. Attack 4: AUC 0.50 to 0.53 with DP and 0.65 without	Pass
Art. 32 Security of processing	AES-256 encryption and SHA-256 manifests	Attack 5: no metadata leakage. Encryption audit passed	Pass

Table 3: GDPR compliance mapping with attack evidence

All images came from CARLA, so we collected no personal data. No human participants were involved, so consent was not needed. Only noise-protected statistics reach the simulator. Every run has a signed audit report. RC produced twelve JSON and PDF reports that name the GDPR articles they cover. KITTI was used only as a utility baseline under its non-commercial license. A pass in Table 3 is a technical result only. The calibration budget under sequential composition (12.6) is above the planned limit of 2.0, so the final legal decision belongs to the data controller.

6. Challenges & Mitigation Strategies

Challenge	Mitigation applied	Alternative for future projects
Keys found in source files before publication	Replaced keys, loaded them from environment variables and added a regression test.	Secret scanning in CI and a key management service.
Proposal specified CBC, which has no tamper detection	Switched to AES-256-GCM and kept the SHA-256 manifest.	Re-encrypt the delivered dataset with GCM.
DP library limits: no batch normalization, unstable Adam, no secure mode	Used group normalization and SGD with momentum. Noted the limit in the certificate.	Use a secure random generator that works with current PyTorch.
Budget reported as 1.8 instead of 7 × 1.8 = 12.6	Corrected the reporting. The noise itself was correct, so no new capture was needed.	Set ε per query near 0.257 so the total stays near 1.8.
Interface mismatches between checkpoints and model architectures	Extracted the state dictionary and used the shared model class.	Publish interface contracts and use public-key signatures.

Table 4: Challenges, mitigation and alternatives

7. Code Review, Audit and Ethical Considerations

Supervisors reviewed our work every sprint. A code review before the full experiment found the exposed keys. RC then audited the PI and DG outputs using separately written code. This found the budget reporting error and reproduced the PI membership inference results to within about 2%. We fixed the issues in the next sprint. On ethics, no real person, vehicle or place was recorded. Files that could reveal source values go only to the auditor. Each thesis states how generative AI was used, as the module policy requires.

8.References

[1] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)," Off. J. Eur. Union, vol. L 119, pp. 1-88, May 2016.

[2] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

[3] Software Engineering Institute, "SEI CERT Coding Standards." [Online]. Available: https://wiki.sei.cmu.edu/confluence/display/seccode

[4] OWASP Foundation, "OWASP Secure Coding Practices - Quick Reference Guide." [Online]. Available: https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

[5] Advanced Encryption Standard (AES), FIPS PUB 197, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2001, updated 2023.

[6] Secure Hash Standard (SHS), FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

[7] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST Spec. Publ. 800-38D, 2007.

[8] E. Barker, "Recommendation for key management: Part 1 - General," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST Spec. Publ. 800-57 Part 1 Rev. 5, 2020.

[9] Information security management systems - Requirements, ISO/IEC 27001:2022, Int. Org. Standardization, Geneva, Switzerland, 2022.

[10] Information Security, Cybersecurity and Privacy Protection: Privacy Enhancing Data De-identification Framework, ISO/IEC 27559:2022, Int. Org. Standardization, Geneva, Switzerland, 2022.

[11] PyCryptodome Project, "PyCryptodome documentation," 2024. [Online]. Available: https://www.pycryptodome.org

[12] A. Yousefpour et al., "Opacus: User-friendly differential privacy library in PyTorch," 2021, arXiv:2109.12298.

[13] Nat. Inst. Standards Technol., "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," U.S. Dept. Commerce, Gaithersburg, MD, USA, Rep. NIST AI 100-1, 2023.